Core01 - Object Method & equals·hashCode 계약

자바의 모든 클래스는 `java.lang.Object`를 상속한다. 그래서 `equals`, `hashCode`, `toString`, `clone`, `getClass` 같은 메소드는 어떤 객체에서든 호출할 수 있다. 문제는 이 메소드들에 **계약(contract)** 이 걸려 있다는 점이다. 계약을 어겨도 컴파일 에러나 예외가 나지 않고, 컬렉션이 조용히 이상하게 동작한다.

1.1 equals

> `equals`는 null이 아닌 참조들 사이에 **동치관계(equivalence relation)** 를 구현해야 한다. 조건은 다섯 가지다.

- **반사성(reflexive)** : null이 아닌 모든 x에 대해 `x.equals(x)`는 참이다.
- **대칭성(symmetric)** : null이 아닌 모든 x, y에 대해 `x.equals(y)`가 참인 것과 `y.equals(x)`가 참인 것은 서로 필요충분조건이다.
- **추이성(transitive)** : `x.equals(y)`가 참이고 `y.equals(z)`가 참이면 `x.equals(z)`도 참이다.
- **일관성(consistent)** : 비교에 쓰는 정보가 바뀌지 않는 한, 몇 번을 호출해도 항상 같은 결과를 반환한다.
- **null 비교** : null이 아닌 모든 x에 대해 `x.equals(null)`은 거짓이다.

동치류 : 동치관계는 원소들을 **동치류(equivalence class)** 로 나눈다. 같은 동치류에 속한 원소끼리는 서로 같고, 서로 대체 가능하다.

- `equals`를 오버라이드하면 대부분 `hashCode`도 오버라이드해야 한다. **equals로 같은 객체는 같은 해시코드를 가져야 한다**는 hashCode의 계약이 있기 때문이다. (1.2에서 자세히 다룬다)
- 인자가 2개인 `Objects.equals(a, b)`(`java.util.Objects`)는 null일 수도 있는 두 참조에 대해서도 동치관계를 구현한다.

Implementation Requirements : Object의 기본 구현

`Object.equals`는 가능한 가장 엄격한 동치관계다. null이 아닌 x, y에 대해 `x == y`, 즉 **같은 객체를 가리킬 때만**true를 반환한다. **참조 동등성** 기준이라서 동치류마다 원소가 딱 하나씩 있다.

```
// java.lang.Object
public boolean equals(Object obj) {
    return (this == obj);
}
```

대체 가능이 equals의 진짜 의미다. equals가 true라는 건 "이 둘 중 아무거나 써도 된다"는 선언이다. `HashSet`이 중복을 버리고 `HashMap`이 키를 덮어쓰는 것도 이 선언을 믿기 때문이다. 그래서 계약을 어기면 예외가 발생하는 것이 아니라 컬렉션이 **조용히** 이상하게 동작한다. equals 버그를 찾기 어려운 이유가 여기에 있다.

조건별로 실제로 깨지는 사례

아래와 같이 `Point`를 상속한 `ColorPoint`가 equals에 color 비교를 추가했다고 하자.

```
class Point {
    final int x, y;

    @Override
    public boolean equals(Object o) {
```

```
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }
}

class ColorPoint extends Point {
    final Color color;

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof ColorPoint cp)) return
false;
        return super.equals(cp) && color == cp.color;
    }
}
```

- **대칭성** : `point.equals(colorPoint)`는 참이다. Point 입장에서는 x, y만 보기 때문이다. 반면 `colorPoint.equals(point)`는 거짓이다. ColorPoint 입장에서 Point는 ColorPoint가 아니기 때문이다.
- **추이성** : 대칭성을 살리려고 Point와 비교할 때는 색을 무시하도록 바꾸면 이번엔 추이성이 깨진다.

```
// ColorPoint.equals — Point와 비교할 때는 색을 무시
if (!(o instanceof Point)) return false;
if (!(o instanceof ColorPoint)) return o.equals(this);
return super.equals(o) && color == ((ColorPoint)
o).color;
```

```
ColorPoint p1 = new ColorPoint(1, 2, Color.RED);
Point      p2 = new Point(1, 2);
ColorPoint p3 = new ColorPoint(1, 2, Color.BLUE);

p1.equals(p2); // true  (색 무시)
```

```
p2.equals(p3); // true  (색 무시)
p1.equals(p3); // false (색 비교) → 추이성 위반
```

게다가 다른 서브클래스(`SmellPoint` 같은)도 같은 방식으로 구현하면 `o.equals(this)`가 서로를 계속 호출해 `StackOverflowError`가 발생할 수도 있다.

- **일관성** : `java.net.URL.equals`는 호스트를 비교하려고 **DNS 조회**를 한다. 그래서 네트워크 상태에 따라 결과가 달라질 수 있다. 공식 문서에서도 이 문제 때문에 URL 대신 `URI`를 쓰라고 권장한다. equals는 메모리에 있는 결정적인 정보만으로 계산해야 한다.

getClass()와 instanceof

① `getClass() != o.getClass()`

- 정확히 같은 클래스끼리만 같다고 판단한다. 그래서 필드를 추가한 서브클래스가 있어도 대칭성과 추이성이 유지된다.
- 대신 필드를 **추가하지 않은** 서브클래스조차 부모와 절대 같을 수 없다. (리스코프 치환 원칙 위반)
- JPA(Hibernate) 프록시처럼 런타임에 생성되는 서브클래스 객체와 비교하면 깨진다.

```
class SpecialPoint extends Point {
    // 필드 추가를 하지 않음
}

Point p = new Point(10, 20);
SpecialPoint sp = new SpecialPoint(10, 20);

// 논리적으로 보면 둘 다 그냥 (10, 20)이다.
// 하지만 getClass()를 사용하면
```

```
p.equals(sp); // false
sp.equals(p); // false
```

즉, equals 규약은 안전하게 지키지만 **상속을 통한 값의 호환성을 포기한** 방식이다. `HashSet<Point>`에 `p`를 넣어두고 `contains(sp)`를 호출하면 false가 나온다.

② `instanceof`

- 서브타입도 대체 가능하다고 취급한다.
- 서브클래스가 equals에 필드를 추가하면 위의 대칭성·추이성 문제가 생긴다.
- `null instanceof X`는 항상 false라서 별도의 null 체크가 필요 없다.

```
@Override
public boolean equals(Object o) {
    if (!(o instanceof Point point)) return false; //
Java 16+ 패턴 매칭
    return x == point.x && y == point.y;
}

// 부모 입장에서는 p.equals(cp) == true 이다.
// 하지만 ColorPoint가 색상까지 비교하도록 equals()를 재정의한다면?
// cp.equals(p) == false 가 된다. → 대칭성 위반
```

> **값 클래스를 상속해서 equals에 필드를 추가하면서 계약을 완벽하게 지키는 방법은 없다.** 그래서 값 클래스는 `final`로 만들고 `instanceof`를 쓰거나, 상속 대신 **조합(composition)** 을 쓴다.

```
// 상속 대신 조합
public final class ColorPoint {
    private final Point point;
    private final Color color;

    public Point asPoint() { return point; } // Point가 필요하면 뷰를 제공
    // equals/hashCode는 point와 color로 구현
}
```

Java 16 이상이라면 `record`를 쓰는 것도 방법이다. record는 암묵적으로 `final`이고, 모든 컴포넌트를 비교하는 equals/hashCode를 자동으로 만들어준다.

흔한 실수

- `public boolean equals(Money o)`는 오버라이드가 아니라 **오버로드**다.
- 컬렉션은 `equals(Object)`를 호출하므로, 오버로드한 메소드는 무시되고 Object의 기본 구현(참조 비교)이 불린다.
- 그래서 `@Override`를 반드시 붙여야 한다. 붙이면 컴파일러가 잡아준다.

```
public boolean equals(Money o) { ... }  // 오버로드 – 컬렉션은 이 메소드를 호출하지 않는다

@Override
public boolean equals(Money o) { ... }  // 컴파일 에러: method does not override
```

필드 타입별 비교 방법

필드 타입	비교 방법	이유
`float`, `double` 제외 원시 타입	`==`	값 비교로 충분하다
`float`, `double`	`Float.compare`, `Double.compare`	`==`로 비교하면 `NaN == NaN`이 false라서 **반사성이 깨진다.** 또 `-0.0 == 0.0`은 true지만 래퍼의 equals는 둘을 다르게 보기 때문에 hashCode와 어긋난다
배열	`Arrays.equals`, `Arrays.deepEquals`(중첩 배열)	배열의 equals는 Object를 그대로 상속한 **참조 비교**다
null이 가능한 필드	`Objects.equals(a, b)`	null 체크를 알아서 해준다
그 외 참조 타입	해당 필드의 `equals`	

1.2 hashCode

`hashCode`는 객체의 해시코드 값을 돌려준다. `HashMap`, `HashSet` 같은 해시 테이블을 위해 존재하는 메소드이며 계약은 다음과 같다.

- 한 번의 애플리케이션 실행 동안 **같은 객체에 여러 번 호출하면 항상 같은 정수를 돌려줘야 한다.** 단, equals 비교에 쓰는 정보가 수정되지 않을 때에 한하며, 실행이 달라지면 값이 달라져도 된다.
- **equals로 같은 두 객체는 반드시 같은 hashCode를 가진다.**
- equals로 다른 두 객체가 다른 hashCode를 가져야 할 의무는 없지만, 다르게 나올수록 해시 테이블 성능이 좋아진다.

조건의 핵심 : HashMap 조회 순서

```
get(key)
 1. key.hashCode()로 해시를 구한다
 2. 상위 16비트를 하위 비트에 섞는다        → hash = h ^ (h >>>
16)
 3. 버킷 인덱스를 계산한다                   → index = (n - 1) &
```

```
hash
 4. 그 버킷 안에서만 탐색한다
 5. 저장된 해시가 같고, (키가 == 이거나 equals가 참)인 노드를 찾는다
```

- 해시가 다르면 **equals는 호출조차 되지 않는다.** 다른 버킷으로 가거나, 같은 버킷이더라도 해시 비교 단계에서 먼저 걸러진다.
- 그래서 equals만 오버라이드하면 논리적으로 같은 객체가 서로 다른 버킷에 들어가고, 결국 중복이 생긴다.

```
// equals만 오버라이드하고 hashCode는 오버라이드하지 않은 경우
Set<Point> set = new HashSet<>();
set.add(new Point(1, 2));
set.contains(new Point(1, 2)); // false — 기본 hashCode가 서로 달라서 다른 버킷을 본다
```

반대로 모든 객체가 같은 해시를 반환하면 계약은 지키지만 전부 한 버킷에 몰린다. Java 8부터는 한 버킷에 노드가 8개 이상(테이블 크기 64 이상)이면 연결 리스트를 트리로 바꿔 최악의 경우를 O(log n)으로 막지만, 해시 테이블의 장점은 사라진다.

조건 1-① : equals에 쓰는 정보가 수정되지 않는 한

정보가 수정되면 해시가 바뀌어도 계약 위반이 아니다. 문제는 `HashMap`이 **넣는 시점의 해시를 노드에 캐싱**한다는 점이다. 그래서 키 객체의 필드를 바꾸면 두 방향 모두에서 못 찾는다.

- 바뀐 객체로 찾으면 새 해시로 계산하니 다른 버킷을 본다.
- 옛 값과 같은 객체로 찾으면 버킷은 맞는데 equals가 false다.

```
MutablePoint key = new MutablePoint(1, 2);
Map<MutablePoint, String> map = new HashMap<>();
map.put(key, "A");
```

```
key.x = 100;                          // 키의 필드 변경
map.get(key);                         // null - 새 해시로 다른
버킷을 본다
map.get(new MutablePoint(1, 2)); // null - 버킷은 맞지만
equals가 false
map.size();                           // 1    - 엔트리는 그대로
남아 있다 (메모리 누수)
```

> 따라서 **해시 키는 불변이어야 한다.**

조건 1-② : 실행이 달라지면 값이 달라도 된다

해시 값을 DB, 파일, 외부 캐시 키로 **영속화하면 안 된다.** 다음 실행에서 다른 값이 나올 수 있다. 참고로 `String.hashCode`는 계산식이 명세에 정의되어 있어 실행마다 같지만, Object의 기본 hashCode나 이를 그대로 쓰는 enum의 hashCode는 실행마다 달라진다.

hashCode는 메모리 주소가 아니다

흔한 오해와 달리 Object의 기본 hashCode(identity hash)는 메모리 주소가 아니다. HotSpot(JDK 8+)은 기본적으로 **스레드 로컬 상태를 이용한 xor-shift 난수**로 값을 생성하고, 처음 호출될 때 객체 헤더(mark word)에 저장한다. GC가 객체를 옮겨도 identity hash는 바뀌면 안 되므로 주소를 그대로 쓸 수 없다. hashCode를 오버라이드한 객체의 identity hash가 필요하면 `System.identityHashCode(o)`를 쓴다.

구현할 때 주의할 점

- equals에 **안 쓰는** 필드를 hashCode에 넣으면 계약 위반이다. equals로 같은 두 객체의 해시가 달라질 수 있기 때문이다.
- 반대로 equals 필드 중 **일부만** hashCode에 쓰는 건 계약상 괜찮지만, 충돌이 늘어날 수 있다.

```java
// 간단하지만 호출마다 varargs 배열 생성 + 박싱 비용이 있다
@Override
public int hashCode() {
    return Objects.hash(x, y);
}

// 성능이 중요하면 직접 계산한다
@Override
public int hashCode() {
    int result = Integer.hashCode(x);
    result = 31 * result + Integer.hashCode(y);
    return result;
}
```

`31`을 곱하는 이유는 홀수 소수라서 분포가 고르고, `31 * i == (i << 5) - i`로 JIT가 시프트 연산으로 최적화할 수 있기 때문이다.

1.3 toString

- toString은 null이 아닌 문자열을 돌려줘야 한다. **객체를 텍스트로 표현한 문자열이어야 하고, 간결하지만 정보가 있고 사람이 읽기 쉬워야 한다.** 그래서 모든 서브클래스에서 오버라이드할 것을 권장한다.
- 출력 형식은 시간이 지나거나 JVM 실행이 바뀌면 달라질 수 있다. 그러니 **toString 결과를 파싱해서 로직에 쓰면 안 된다.**
- 기본 구현은 `getClass().getName() + '@' + Integer.toHexString(hashCode())`이다.
  (예: `com.example.Point@1b6d3586`)
- toString과 hashCode를 둘 다 오버라이드했는데 기본 구현과 같은 문자열이 필요하면 `Objects.toIdentityString(o)`(Java 19+)를 쓴다.

기본 구현은 내부에서 `hashCode()`를 호출한다. 그래서 **hashCode만 오버라이드해도 기본 toString 출력이 바뀐다.**

toString은 **양방향 참조 구조**에서 특히 주의해야 한다. 서로의 toString을 부르면 무한 재귀로 `StackOverflowError`가 발생한다. JPA 엔티티에 Lombok `@ToString`이나 `@Data`를 붙였을 때 자주 터지는 문제다.

```
@Entity
class Team {
    @OneToMany(mappedBy = "team")
    private List<Member> members;

    @Override
    public String toString() { return "Team{members=" +
members + "}"; }
}

@Entity
class Member {
    @ManyToOne
    private Team team;

    @Override
    public String toString() { return "Member{team=" +
team + "}"; }
}

// team.toString() → members.toString() →
member.toString() → team.toString() → ...
// → StackOverflowError
```

연관 필드는 toString에서 빼거나(Lombok이면 `@ToString.Exclude`) ID만 출력한다. 지연 로딩 필드를 toString에서 건드리면 의도치 않은 쿼리가 나가는 문제도 함께 막을 수 있다.

1.4 clone

객체의 사본을 만들어 돌려준다. 일반적인 의도는 다음 세 가지이며, 절대적인 요구사항은 아니다.

- `x.clone() != x` : 다른 객체여야 한다.
- `x.clone().getClass() == x.getClass()` : 같은 클래스여야 한다.
- `x.clone().equals(x)` : 내용이 같아야 한다.

반환 객체는 관례상 `super.clone()`을 호출해서 얻어야 한다. 클래스와 모든 상위 클래스(Object 제외)가 이 관례를 따르면 `getClass()`가 같다는 것이 보장된다.

반환 객체는 원본과 **독립적**이어야 한다. 그러려면 `super.clone()`으로 받은 객체의 필드 일부를 고쳐야 할 수 있다. 보통은 객체 내부의 **깊은 구조**를 이루는 **가변 객체들을 복사해서 참조를 바꿔 끼우는 작업**이다. 필드가 원시 타입이거나 불변 객체 참조면 고칠 필요가 없다.

```
class Stack implements Cloneable {
    private Object[] elements;
    private int size;

    @Override
    public Stack clone() {                          // 공변 반환 타입으로 캐스팅 부담을 없앤다
        try {
            Stack result = (Stack) super.clone(); // 여기까지는 얕은 복사
            result.elements = elements.clone();   // 가변 배열은 따로 복사해서 바꿔 끼운다
            return result;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();           //
```

```
            Cloneable을 구현했으니 일어날 수 없다
        }
    }
}
```

`elements.clone()`을 빼먹으면 원본과 사본이 같은 배열을 공유해서, 한쪽에 push하면 다른 쪽이 깨진다.

Object.clone의 기본 동작

- 클래스가 `Cloneable`을 구현하지 않았으면 `CloneNotSupportedException`을 던진다.
- 모든 배열은 `Cloneable`을 구현한 것으로 간주되고, `T[]`의 clone 반환 타입은 `T[]`다.
- 그 외에는 새 인스턴스를 만들고 모든 필드를 대입하듯이 복사한다. 필드가 가리키는 객체 자체는 복제하지 않으므로 **얕은 복사(shallow copy)** 다.
- Object 자체는 `Cloneable`이 아니라서, 클래스가 딱 Object인 객체에 clone을 호출하면 런타임 예외가 발생한다.

clone을 피하는 이유

- `Cloneable`은 메소드가 하나도 없는 인터페이스인데, 부모(Object)에 있는 `protected` 메소드의 동작을 바꾼다. 구현하면 필드를 복사한 객체를 반환하고, 구현하지 않으면 예외를 던진다. 인터페이스를 쓰는 아주 이례적인 방식이다.
- **생성자를 거치지 않고** 객체가 생겨서, 생성자에 넣은 불변식 검증이 무시된다.
- **final 필드와 충돌한다.** 깊은 복사를 하려면 clone 안에서 필드를 재할당해야 하는데, 가변 객체를 참조하는 필드가 final이면 그게 안 된다.

- `protected`이고 검사 예외(`CloneNotSupportedException`)를 던지기 때문에 사용하기 불편하다.

대안은 **복사 생성자**나 **복사 팩토리**다. `new ArrayList<>(list)`, `List.copyOf(list)`처럼 생성자를 거치니 불변식이 지켜지고, final 필드도 문제없다. 단, 배열은 clone이 가장 깔끔한 복사 방법이라 예외로 써도 된다.

1.5 getClass

`getClass()`는 객체의 **런타임 클래스**를 돌려준다. 반환되는 Class 객체는 그 클래스의 `static synchronized` 메소드가 잠그는 바로 그 객체다.

실제 결과 타입은 `Class<? extends |X|>`다. 여기서 `|X|`는 getClass를 호출한 식의 **정적 타입을 erasure한 것**이다.

```
Number n = 0;
Class<? extends Number> c1 = n.getClass();   // 정적 타입은 Number, 실제 값은 Integer.class

List<String> list = new ArrayList<>();
Class<? extends List> c2 = list.getClass();  // 정적 타입 List<String>의 erasure인 List
```

- `final` 메소드라서 오버라이드할 수 없다.
- 같은 클래스 로더 안에서 Class 객체는 하나뿐이라 `==`로 비교해도 된다. 1.1의 `getClass() != o.getClass()` 비교가 가능한 이유다.
- Hibernate 프록시 객체에 getClass를 호출하면 엔티티 클래스가 아니라 **프록시 클래스**가 나온다. 엔티티 equals에서 getClass 비교가 깨지는 원인이다.

1.6 java.util.Objects 메소드들

`java.util.Objects`는 Java 7에 추가된 유틸리티 클래스로, Object 메소드를 **null-safe**하게 호출하거나 null을 검증하는 static 메소드를 모아둔 것이다.

메소드	동작	도입
`equals(a, b)`	`a == b`면 true(둘 다 null인 경우 포함), a가 null이면 false, 그 외에는 `a.equals(b)`	7
`deepEquals(a, b)`	같은 참조면 true, 하나만 null이면 false. 둘 다 배열이면 `Arrays.deepEquals`처럼 원소를 재귀 비교(원시 타입 배열 포함), 아니면 `a.equals(b)`	7
`hashCode(o)`	null이면 0, 아니면 `o.hashCode()`	7
`hash(values...)`	`Arrays.hashCode(values)`. 여러 필드로 해시를 만들 때 쓴다	7
`toString(o)`	`String.valueOf(o)`. null이면 `"null"`	7
`toString(o, nullDefault)`	null이면 `nullDefault`, 아니면 `o.toString()`	7
`requireNonNull(obj[, msg / supplier])`	null이면 `NullPointerException`, 아니면 obj를 그대로 반환. Supplier 버전은 null일 때만 메시지를 생성한다	7 (Supplier는 8)
`requireNonNullElse(obj, defaultObj)`	obj가 null이면 defaultObj 반환. defaultObj도 null이면 NPE	9
`requireNonNullElseGet(obj, supplier)`	obj가 null이면 `supplier.get()` 반환. 기본값을 **지연 생성**한다	9
`isNull(obj)` / `nonNull(obj)`	`obj == null` / `obj != null`. `Predicate` 메소드 참조용	8
`compare(a, b, comparator)`	`a == b`면 0, 아니면 `comparator.compare(a, b)`	7
`checkIndex(index, length)`	`0 <= index < length`가 아니면 `IndexOutOfBoundsException`, 맞으면 index 반환	9
`toIdentityString(o)`	Object 기본 toString과 같은 문자열 (1.3 참고)	19

```
// 생성자 파라미터 검증 – null 체크와 필드 대입을 한 줄로
this.name = Objects.requireNonNull(name, "name must not
```

```
be null");

// 메시지 조립 비용이 크면 Supplier로 지연 생성 (null이 아니면 문자열을 만들지 않는다)
Objects.requireNonNull(user, () -> "user not found: " + id);

// null이면 기본값으로 대체
String nickname = Objects.requireNonNullElse(input, "anonymous");

// 스트림에서 null 걸러내기
List<String> names = list.stream().filter(Objects::nonNull).toList();
```

> **주의** : `Objects.hash(x)`와 `Objects.hashCode(x)`는 결과가 다르다. `hash`는 인자를 배열로 받아 `Arrays.hashCode`를 계산하므로 `31 + x.hashCode()`가 나온다. 필드가 하나뿐이라면 `hashCode(x)`를 쓰는 게 맞다.